UNIVERSITÉ DE SHERBROOKE
Faculté de génie
Département de génie électrique et génie informatique

LOG BOOK

Projet S5

Présenté à
Domingo Palao Munoz

Présenté par
Équipe 1

Sherbrooke – 23 septembre 2026

Table des matières

1 Définition du projet 1
1.1 Planification du projet 1
1.2 Signatures 1
2 Architecture du système embarqué 2
2.1 Présentation 2
2.2 Communication 2
2.3 Signatures 2
3 Système Linux embarqué 4
3.1 Présentation 4
3.2 PID 1 4
3.3 Paquets ajoutés 4
3.4 Signatures 4
4 Avancements et modification 6
4.1 Avancements 6
4.2 Modification 6
4.3 Signatures 6
5 Formules GoDot 8
5.1 Présentation 8
5.2 Signatures 8
6 Avancements et modification 9
6.1 Avancements 9
6.2 Simulation GoDot (Raphaël Bonhomme) 9
6.3 Modification 9
6.4 Signatures 9
7 Avancements 11
7.1 Capteurs de ligne analogiques (Raphaël Bonhomme) 11
7.2 Roues animées 11
7.3 Signatures 11
8 Avancements 12
8.1 Capteurs de distance (Jonathan Valade) 12
8.2 Shared memory Region (Marc-Antoine Gauthier) 12
8.3 Fichiers de configuration 12
8.4 Signatures 12
9 Avancements 14
9.1 Mode réaliste et obstacles dans GoDot (Raphaël Bonhomme) 14
9.2 Évitement d'obstacle, première version 14
9.3 Déploiement sur le robot 14
9.4 Fusion du code et correctifs de la simulation 14
9.5 Signatures 15
10 Avancements 16
10.1 FSM (Jonathan Valade) 16

10.2 Configuration variable (Tristan Faucher) 16
10.3 Outils d'interprétation 16
10.4 Signatures 16
11 Avancements 18
11.1 Évitement d'obstacle simplifié (Raphaël Bonhomme) 18
11.2 Tests accélérés de la simulation 18
11.3 Réglages sans recompiler 18
11.4 Arrêt d'urgence 18
11.5 Puissance maximale 19
11.6 Signatures 19
12 Avancements 20
12.1 Arrêt à l'arrivée et protection de la bille (Raphaël Bonhomme) 20
12.2 Évitement d'obstacle en 12 étapes 20
12.3 Validation en simulation et problèmes ouverts 21
12.4 Documentation 21
12.5 Signatures 21

Liste des tableaux

Table des figures

1 Définition du projet

Date de décision : 6 octobre 2026
Proposé par : L'Équipe
Statut : Adopté à l'unanimité
Absences : Aucune

1.1 Planification du projet

Afin de planifier le projet de manière adéquate, l'équipe c'est rencontré pour séparer les parties à écrire du rapport *Vends-moi ton breuvage*.

De cette manière, les définitions des tâches et réalisations dans le projet représentent le travail à réaliser par l'équipe pour relever les défis de conceptions du client.

Une évaluation des risques à aussi été présenté dans ledit document tel que convenu avec le client.

1.2 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

2 Architecture du système embarqué

Date de décision : 15 octobre 2026
Proposé par : Marc-Antoine Gauthier
Statut : Adopté à l'unanimité
Absences : Aucune

2.1 Présentation

Afin de répondre au besoin du projet, une architecture proposant l'utilisation de plusieurs services représentant une composante unitaire du robot serait souhaitable. De ce fait, l'architecture serait basée sur 4 services principaux.

FSM Permet de prendre des décisions sur l'exécution autonome du robot en fonction des données entrantes.

Sensors Permet l'acquisition des données sensoriels du robots

Motors Envoie les commandes aux moteurs propulseurs ainsi qu'au servo moteur pour la direction

Telemetry Récupère les informations du robot et envoient une copie à un client externe via *TCP*. Permet aussi la réception des commandes.

Au besoin, l'utilisation d'une architecture de ce type permetterait d'ajouter sans enjeux réels d'autres services pour les besoins du projet.

2.2 Communication

En ce qui concerne l'envoie de données entre les services, l'architecture prévoie utiliser deux types de communication IPC

POSIX Socket Envoie des commandes entre les services.

Shared Memory Region Partage des données télémétriques et commandes ayant une contrainte de temps (*Real Time Processing*)

2.3 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

3 Système Linux embarqué

Date de décision : 15 octobre 2026
Proposé par : Marc-Antoine Gauthier
Statut : Adopté à l'unanimité
Absences : Aucune

3.1 Présentation

Afin d'accélérer la vitesse d'acquision et apprendre d'avantage sur l'utilisation de Linux en système embarqué, La proposition suivante utiliserait un système minimal *Buildroot* pour pouvoir compiler et déployer une distribution Linux.

3.2 PID 1

Contrairement à *Raspberry PI OS*, le système d'exploitation aurait un programme d'entré minimal sous forme de service. Celui-ci serait responsable d'attacher les bibliothèques et les pilotes nécessaires pour les sous services approuvé au point 1. De plus, Ce service doit démarrer les services et s'assurer de redémarrer ceux ayant terminés. En aucun cas, le **ServiceManager** doit sortir de la boucle. Autrement, le kernel Linux lancera une erreur et obligera un redémarrage du système.

3.3 Paquets ajoutés

wpa_supplicant Permettera d'utiliser le module WIFI et intéragir avec le pilote

libgpiod2 Communiquera avec le pilote responsable des *GPIO*

i2c-dev Intéragira avec les contrôlleurs I^2C pour les moteurs et le capteur de ligne.

sh Exécution des scripts *shell* dans le terminal

dropbear Établiera une communication SSH avec l'utilisation du port Ethernet statique ou du WIFI

3.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

4 Avancements et modification

Date de décision : 2 novembre 2026
Proposé par : Jonathan Valade
Statut : Adopté à l'unanimité
Absences : Aucune

4.1 Avancements

Au cours de la journée, l'architecture embarquée a majoritairement été commencé pour retrouver les services principaux. L'utilisation d'un seul *Shared memory region* pour représenter le robot pour tout les services à été choisi pour des questions de simplicités. Le système Ethernet permettait d'entrer sur le robot de manière fiable et efficace. En utilisant un utilitaire *SCP* pour envoyer les services à des fins de déverminage, l'exécution de *scripts* permettant d'automatiser le deployement de l'environement de test permettait de rapidement avoir un prototype testable. Il a été possible d'évaluer des constantes pour calibrer le capteur de ligne afin que le robot puisse normaliser et moyenner, par le biais d'algorithmes, une ligne.

4.2 Modification

Afin de démarrer le robot et le WIFI sans à avoir à utiliser le cable ethernet, la proposition décrit l'utilisation de deux boutons et l'intégration d'un service **PeripheralsService** pour détecter l'intéraction avec les boutons et exécuter l'action suivant lesdites intéractions. De plus, un système sonore permetterais de confirmer que le robot est démarré ou bien de connaître l'état des systèmes (WIFI ouvert/fermé, démarrage de la séquence, etc.)

4.3 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

5 Formules GoDot

Date de décision : 2 novembre 2026
Proposé par : Jérémie Perron
Statut : Adopté à l'unanimité
Absences : Aucune

5.1 Présentation

Le rapport d'équations *Filtre tes connaissances* utilisé pour la simulation GoDot est disponible. Les équations présentées dans le rapport seront validées par simulation avant d'être implémentées dans le robot.

5.2 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

6 Avancements et modification

Date de décision : 3 novembre 2026
Proposé par : Tristan Faucher
Statut : Adopté à l'unanimité
Absences : Aucune

6.1 Avancements

Le développement d'aujourd'hui a majoritairement de finaliser les services de base et de commencer la télémétrie. Des outils à utilisation secondaire au projet ont aussi été réalisé avec l'assistance d'intelligence artificielle pour alégé le temps de développement. Le service responsable des décisions devrait fonctionner au plus tard demain lorsque les simulations GoDot seront terminé.

6.2 Simulation GoDot (Raphaël Bonhomme)

La simulation présentait quelques imperfection vis-à-vis l'accélération dans les courbes. De plus, le robot simulé allait beaucoup trop droit comparativement à celui utilisé dans le projet. Une solution simple a été d'implémenter des imperfections dans la simulation afin que le modèle subisse des forces extérieurs pour nuire au mouvement.

6.3 Modification

Pour optimiser le déploiement des constantes entre la simulation et GoDot, la proposition suivante recommande d'utiliser un service tier qui connectera GoDot et les services embarqués. L'objectif est de placer les services dans un container *Docker* afin de connecter localement GoDot aux services.

6.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

7 Avancements

Date de décision : 3 novembre 2026
Proposé par : Raphaël Bonhomme
Statut : Adopté à l'unanimité
Absences : Aucune

7.1 Capteurs de ligne analogiques (Raphaël Bonhomme)

Chaque capteur de ligne de la simulation GoDot lance maintenant 15 rayons verticaux répartis sur ses 19 mm de largeur. La fraction de rayons qui touche le ruban donne une valeur continue de 0 (blanc) à 1 (noir), comme un capteur lu par un convertisseur analogique. Cela évite les à-coups de direction qu'occasionne une lecture tout ou rien, où la position du ruban saute par pas de 19 mm.

7.2 Roues animées

Les quatre roues du modèle 3D roulent selon la distance parcourue et les roues avant braquent avec l'angle de direction. On voit donc à l'écran un éventuel défaut d'alignement des roues.

7.3 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

8 Avancements

Date de décision : 4 novembre 2026
Proposé par : Tristan Faucher
Statut : Adopté à l'unanimité
Absences : Aucune

8.1 Capteurs de distance (Jonathan Valade)

Le service responsable des capteurs possède maintenant la capacité de détecter des objets en utilisant le capteur ultrason. Celui-ci utilise les broches *GPIO* pour envoyer une ondes et attendre la réception.

Un rapport détaillé expliquant les fondements et modélisation mathématique est disponible avec le nom : *Calcule tes distances*.

8.2 Shared memory Region (Marc-Antoine Gauthier)

Au lieu d'utiliser une seule région partagée entre les services, le système utilise maintenant une région par service pour le transfert sans délai d'information. Cela allège les changements de contextes et isole correctement les fichiers mémoires des services.

8.3 Fichiers de configuration

Afin de ne pas à avoir à recompiler les services lors d'un changement de paramètre, le système prend en charge l'utilisation de fichiers pour configurer le service. Modifiable par le service de *Télémétrie* via des **POSIX sockets**, cette amélioration simplifiera les modifications externes par le biais d'un utilitaire externe.

8.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

9 Avancements

Date de décision : 4 novembre 2026
Proposé par : Raphaël Bonhomme
Statut : Adopté à l'unanimité
Absences : Aucune

9.1 Mode réaliste et obstacles dans GoDot (Raphaël Bonhomme)

Le mode réaliste, décrit dans l'entrée du 3 novembre, est ajouté au dépôt avec ses réglages. Il ajoute à la simulation les imperfections attendues du robot réel : roues mal alignées (biais de 4°), jeu du servo de direction (2°), moteur à 93 % de la vitesse demandée, latence de 60 ms, petits chocs aléatoires, bruit sur le capteur ultrason et erreurs occasionnelles sur le capteur de ligne. Il s'active avec la touche `R` de GoDot, et le hasard est tiré avec une graine fixe, ce qui rend un parcours reproductible.

Les cubes de la simulation ont maintenant une collision : la voiture ne peut plus les traverser et le capteur ultrason les détecte.

9.2 Évitement d'obstacle, première version

Le FSM s'arrête devant l'obstacle, recule, le contourne par un côté puis retrouve la ligne. Le côté du contournement se choisit au lancement de la simulation avec `./run_sim.sh -direction gauche` ou `droite`. La manœuvre tient compte de la vitesse réelle du robot, environ 0,32 m/s à 50 % de puissance, et non de la vitesse interne du FSM.

9.3 Déploiement sur le robot

Deux scripts ont été ajoutés. Le premier, `build.sh -embedded`, compile les services pour le Raspberry Pi (ARM64) dans un conteneur *Docker*. Le second, `deploy.sh`, envoie les binaires par *SSH*, sauvegarde les anciens sur le robot et peut lancer le robot.

9.4 Fusion du code et correctifs de la simulation

La fusion avec le code refactorisé a laissé des erreurs de compilation, corrigées dans la journée. La simulation a ensuite révélé des fonctions perdues pendant la refonte, qui ont été rétablies :

- Le robot ne démarrait plus en simulation : le tube de démarrage n'était plus créé. Le script `run_sim.sh` le crée maintenant.
- L'évitement échouait de façon aléatoire : le FSM tournait à 100 Hz alors que les gains sont réglés pour 50 Hz. Le fichier `fsm.conf` fixe maintenant 50 Hz.
- Le choix du côté de contournement était ignoré : la variable `AVOID_SIDE` n'était plus lue.
- Le ralentissement à l'approche de l'obstacle avait disparu : le robot arrivait à pleine vitesse et se coinçait contre le cube. Il est rétabli et vérifié par un test unitaire.

- Le robot corrigeait ses erreurs dans le mauvais sens : l'ordre des cinq capteurs de ligne n'était plus inversé entre GoDot et le robot (GoDot est en miroir de la convention du robot).

9.5 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

10 Avancements

Date de décision : 5 novembre 2026
Proposé par : Jonathan Valade
Statut : Adopté à l'unanimité
Absences : Aucune

10.1 FSM (Jonathan Valade)

Afin de permettre une meilleure gestion de la FSM, les états ont été retravaillés afin de retirer le mode **Reculon** lorsque le robot perd la ligne.

Cette amélioration permet d'éviter des accélérations et désaccélérations inutiles lors de la perte de la ligne.

10.2 Configuration variable (Tristan Faucher)

Les modifications apportées à la bibliothèque partagée sont finalisé et un test à été effectué avec le client externe.

10.3 Outils d'interprétation

Afin de simplifier le post-traitement des informations, reçu par le robot, L'équipe c'est équipé d'outils rapidement développé par l'IA afin de lire la télémétrie en temps réel et modéliser les actions prises par le robot après l'utilisation de celui-ci.

Quelques problèmes ont été noté vis-à-vis les programmes fabriqués. Ceux-ci se sont réglé en analysant correctement la logique du code.

10.4 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

11 Avancements

Date de décision : 5 novembre 2026
Proposé par : Raphaël Bonhomme
Statut : Adopté à l'unanimité
Absences : Aucune

11.1 Évitement d'obstacle simplifié (Raphaël Bonhomme)

La manœuvre d'évitement a été refaite en cinq phases simples, avec des angles et des durées fixés à l'avance : reculer, s'écarter (virage puis diagonale), se redresser (virage puis ligne droite pour dépasser le cube), revenir (virage vers la ligne), puis avancer jusqu'à retrouver le ruban. Elle fonctionne en boucle ouverte : seule la dernière phase consulte les capteurs de ligne.

Les durées ont été calibrées en simulation par des balayages de plusieurs dizaines de combinaisons, jusqu'à trouver une zone où l'évitement réussit des deux côtés. Les valeurs retenues sont au centre de cette zone, ce qui évite de dépendre d'un seul réglage chanceux. Le deuxième obstacle est placé à environ 0,6 m d'un coin à 90° : si la manœuvre avance trop longtemps, le robot arrive trop près du coin et la manœuvre échoue du côté droit.

11.2 Tests accélérés de la simulation

GoDot peut s'exécuter sans fenêtre et être piloté par des scripts de test qui relèvent la position du robot. Pour aller plus vite que le temps réel, il faut accélérer ensemble GoDot (temps et nombre de pas de physique) et la boucle du FSM, avec le même pas de temps logique. Un tour complet passe ainsi d'environ 53 s à 16 s à une vitesse de 5 fois. À 8 fois, le FSM ne suit plus. Ces scripts ne font pas encore partie du dépôt.

11.3 Réglages sans recompiler

Les angles, les durées et les vitesses de l'évitement sont des clés du fichier `fsm.conf`. Le FSM borne chaque valeur pour éviter un réglage dangereux. Les clés se modifient aussi à chaud depuis le site de télémétrie, dans une section dédiée à l'évitement d'obstacle, et le FSM écrit la nouvelle valeur dans le fichier.

11.4 Arrêt d'urgence

Le bouton d'arrêt d'urgence du site de télémétrie n'était envoyé qu'au service `Motors`, qui l'appliquait une seule fois. Le FSM n'était pas prévenu : il continuait à commander les moteurs et le robot repartait au cycle suivant. Le client envoie maintenant l'arrêt au FSM puis à `Motors`. Le FSM passe en état `Abort`, maintient la puissance à zéro et ne repart qu'après un nouvel ordre de démarrage.

Un test automatisé vérifie que les deux ordres sont émis. En simulation, le robot reste immobile après l'arrêt (dérive de 0,000 m) et repart après le réarmement.

11.5 Puissance maximale

La conversion entre vitesse et puissance moteur est puissance = (vitesse/croisière) × 50 %. La vitesse de croisière n'est donc pas une vitesse réelle : la voiture est toujours à 50 % de puissance (environ 0,32 m/s) à cette vitesse, quelle que soit sa valeur. Un nouveau paramètre, `MAX_POWER_PCT` (50 % par défaut, borné de 20 à 75 %), permet de choisir cette puissance. La vitesse de croisière est maintenant bornée entre 0,10 et 0,60, ce qui évite une valeur nulle ou négative. Une puissance plus élevée augmente le risque pour la bille : elle n'a pas été testée.

11.6 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade

12 Avancements

Date de décision : 11 novembre 2026
Proposé par : Raphaël Bonhomme
Statut : Adopté à l'unanimité
Absences : Aucune

12.1 Arrêt à l'arrivée et protection de la bille (Raphaël Bonhomme)

La bille repose dans une coupelle conique très aplatie : rayon de 20 mm et profondeur de 1,5 mm, soit une pente $\tan\theta = 1{,}5/20 = 0{,}075$. La bille sort de la coupelle lorsque l'accélération du robot dépasse l'accélération critique :

$$a_{max} = g\tan\theta = 9{,}81 \times 0{,}075 \approx 0{,}735 \text{ m/s}^2$$

Le FSM la respecte avec un profil de vitesse qui limite la variation de vitesse à a_{max} par seconde, et avec une vitesse maximale en virage $v = k\sqrt{a_{max}\,R}$, où k est un facteur de sécurité et $R = L/\tan\delta$ est le rayon du virage ($L = 0{,}14$ m est l'empattement et δ l'angle des roues).

En simulation, la bille tombait pourtant dès que le robot atteignait la barre d'arrivée. La cause était une coupure instantanée de la puissance moteur dans l'état `Stopped` : la vitesse passait de 0,32 m/s à zéro en un seul cycle de 20 ms, soit environ 16 m/s^2, plus de vingt fois la limite de la bille.

La correction réserve la coupure immédiate aux vrais arrêts d'urgence et aux échecs. À l'arrivée, la vitesse suit maintenant le profil de freinage : le robot s'arrête en environ 0,43 s et quelques centimètres après la barre. Un test automatisé vérifie que la puissance ne baisse jamais de plus d'environ 2,3 points par cycle. En simulation, la bille tombait dans 2 essais sur 3 avant la correction et dans aucun des 3 essais après.

12.2 Évitement d'obstacle en 12 étapes

La manœuvre a été simplifiée en une suite d'étapes précises, où chaque angle et chaque durée est réglable. Les valeurs ci-dessous sont celles par défaut, validées en simulation.

1. Reculer de 0,25 m.
2. Tourner les roues de 45° du côté choisi.
3. Avancer 1,10 s.
4. Mettre les roues droites.
5. Avancer 1,10 s (diagonale qui écarte le robot du cube).
6. Tourner les roues de 45° de l'autre côté.
7. Avancer 0,70 s.

8. Mettre les roues droites et avancer 1,00 s : le robot est parallèle à la ligne et dépasse le cube.
9. Tourner les roues de 45° vers la ligne.
10. Avancer 0,70 s.
11. Mettre les roues droites.
12. Avancer jusqu'à retrouver la ligne (durée maximale de 3,5 s).

À 0,16 m/s, une seconde de déplacement correspond à 16 cm. Un virage de 45° avec les roues à fond tourne le cap d'environ 45° : allonger la durée d'un virage augmente la rotation du robot. Chaque angle, chaque durée et chaque vitesse est une clé `AVOID_*` du fichier `fsm.conf`, bornée par le FSM et modifiable depuis le site de télémétrie. Les durées supposent la vitesse de manœuvre de 0,16 m/s, soit 25 % de la puissance moteur lorsque la vitesse de croisière est à 0,32 : si la croisière change, il faut calibrer de nouveau.

12.3 Validation en simulation et problèmes ouverts

- Premier obstacle : contourné et ligne retrouvée dans les quatre cas testés (droite et gauche, avec et sans le mode réaliste).
- Deuxième obstacle seul, sans le mode réaliste : réussi des deux côtés.
- Avec le mode réaliste, le robot perd la ligne au premier virage de la piste, même sans obstacle. Le suivi de ligne est en cause, et ni la vitesse ni les gains testés ne règlent le problème.
- Depuis la simplification de la machine à états, des tours complets s'arrêtent parfois à tort en cours de parcours (la barre d'arrivée est détectée trop tôt).
- Les réglages de l'évitement et du freinage n'ont pas été validés sur le robot réel.

12.4 Documentation

Une documentation de la simulation a été rédigée : lancement, scène GoDot, capteurs de ligne et de distance, communication avec le FSM, suiveur de ligne, bille, mode réaliste, tests et dépannage.

12.5 Signatures



Marc-Antoine Gauthier



Jérémie Perron



Tristan Faucher



Raphaël Bonhomme



Jonathan Valade